ASSIGNMENT NO. 5

AIM: Implement the C program for Deadlock Avoidance Algorithm: Bankers Algorithm.

Description: Banker's Algorithm in Operating System

The banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation for predetermined maximum possible amounts of all resources, then makes an "s-state" check to test for possible activities, before deciding whether allocation should be allowed to continue.

Banker's algorithm is used in banking system to check whether loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. If a person applies for a loan then the bank first subtracts the loan amount from the total money that bank has and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders comes to withdraw their money then the bank can easily do it. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers. The bank would try to be in safe state always.

Data structures are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resources types.

1) Available:

- It is a 1-d array of size **'m'** indicating the number of available resources of each type.
- Available [j] = k means there are **'k'** instances of resource type **Rj**

2) Max :

- It is a 2-d array of size '**n*m'** that defines the maximum demand of each process in a system.
- Max[i, j] = k means process **Pi** may request at most **'k'** instances of resource type **Rj.**

3) Allocation :

- It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
- Allocation[i, j] = k means process **Pi** is currently allocated **'k'** instances of resource type **Rj**

4) Need :

- It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
- Need [i, j] = k means process **Pi** currently need **'k'** instances of resource type **Rj**
- Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation i specifies the resources currently allocated to process Pi and Needi specifies the additional resources that process Pi may still request to complete its task. Banker's algorithm consists of Safety algorithm and Resource request algorithm

Safety Algorithm
The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1) Let Work and Finish be vectors of length 'm' and 'n' respectively.
Initialize: Work = Available
Finish[i] = false; for i=1, 2, 3, 4….n
2) Find an i such that both
a) Finish[i] = false
b) Need i <= Work
if no such i exists goto step (4)
3) Work = Work + Allocation[i]
Finish[i] = true
goto step (2)
4) if Finish [i] = true for all i
then the system is in a safe state

Resource-Request Algorithm
Let Request i be the request array for process Pi. Request i [j] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1) If Request i <= Need i
Goto step (2) ; otherwise, raise an error condition, since the process has exceeded its maximum claim.
2) If Request i <= Available
Goto step (3);
otherwise, Pi must wait, since the resources are not available.
3) Have the system pretend to have allocated the requested resources to process Pi by modifying the state as
follows:
Available = Available – Request i
Allocation i = Allocation i + Request i
Need i = Need i– Request i

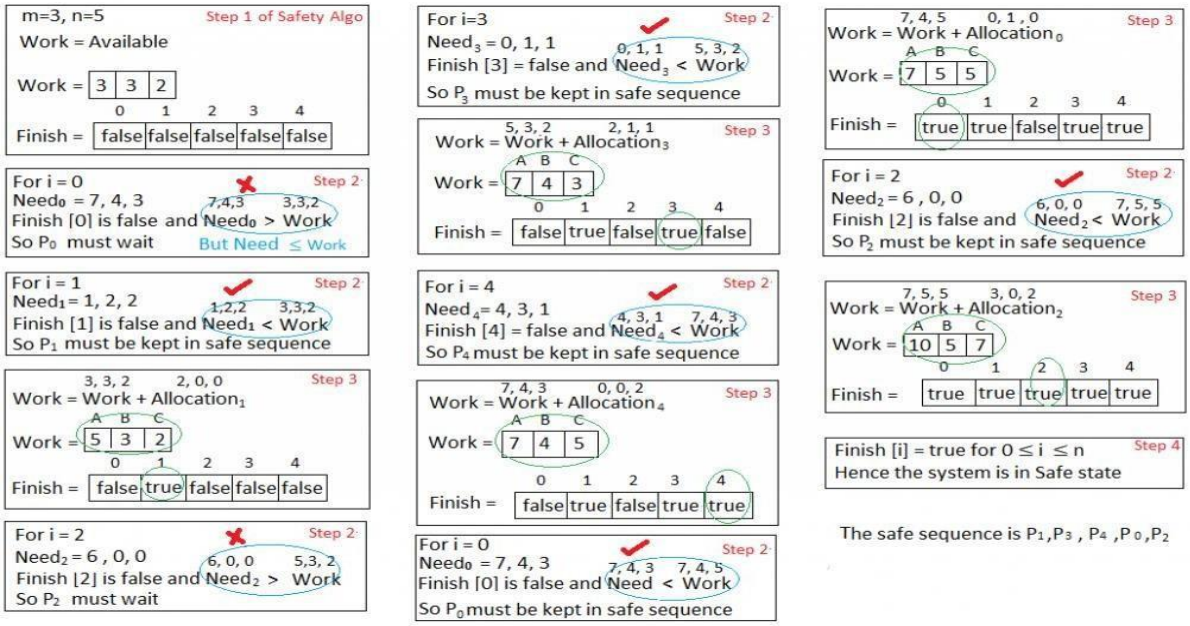
Example:
Considering a system with five processes P0 through P4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Question1. What will be the content of the Need matrix?
Need [i, j] = Max [i, j] – Allocation [i, j]
So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Question2. Is the system in a safe state? If Yes, then what is the safe sequence?
Applying the Safety algorithm on the given system,



Program 5:

```
#include <stdio.h>

#define MAX 10

int main()
{
    int n, m;
    int allocation[MAX][MAX];
    int maximum[MAX][MAX];
    int need[MAX][MAX];
    int available[MAX];
    int work[MAX];
    int finish[MAX] = {0};
    int safeSequence[MAX];
    int count = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    /* Input Allocation Matrix */
    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        printf("P%d: ", i);
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    /* Input Maximum Matrix */
    printf("\nEnter Maximum Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        printf("P%d: ", i);
        for (int j = 0; j < m; j++)
        {
            scanf("%d", &maximum[i][j]);
        }
    }

    /* Input Available Resources */
    printf("\nEnter Available Resources:\n");
    for (int j = 0; j < m; j++)
    {
        scanf("%d", &available[j]);
        work[j] = available[j];
    }

    /* Calculate Need Matrix */
    for (int i = 0; i < n; i++)
```

```
{
    for (int j = 0; j < m; j++)
    {
        need[i][j] = maximum[i][j] - allocation[i][j];
    }
}

/* Display Need Matrix */
printf("\nNeed Matrix:\n");
for (int i = 0; i < n; i++)
{
    printf("P%d: ", i);
    for (int j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

/* Banker's Safety Algorithm */
while (count < n)
{
    int found = 0;

    for (int i = 0; i < n; i++)
    {
        if (finish[i] == 0)
        {
            int possible = 1;

            /* Check Need <= Work */
            for (int j = 0; j < m; j++)
            {
                if (need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if (possible)
            {
                /* Process can complete */
                for (int j = 0; j < m; j++)
                {
                    work[j] += allocation[i][j];
                }

                safeSequence[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    /* No process can proceed */
    if (found == 0)
    {
        break;
```

```
        }
    }

    /* Check Safe State */
    if (count == n)
    {
        printf("\nSystem is in SAFE STATE.\n");

        printf("Safe Sequence: ");

        for (int i = 0; i < n; i++)
        {
            printf("P%d", safeSequence[i]);

            if (i != n - 1)
                printf(" -> ");
        }

        printf("\n");
    }
    else
    {
        printf("\nSystem is in UNSAFE STATE.\n");
        printf("Deadlock may occur.\n");
    }

    return 0;
}
```